

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Coding Challenge)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-CODING CHALLENGE
Question	Q3 — Challenge 3: String Compression and Run-Length Encoding (Medium 15 M

Challenge 3: String Compression and Run-Length Encoding (RLE) [Level: Medium | 15 Marks]

Implement a Run-Length Encoding (RLE) program that: (a) compresses a string by replacing consecutive repeated characters with character+count, (b) decompresses the encoded string back to the original, and (c) if the compressed string is not shorter, returns the original.

Constraints

- Single characters without count still output as single char.
- Handle empty strings.
- Decompressor should handle all valid RLE strings.

Skills Assessed

- String traversal and comparison
- Count accumulation logic
- String building and integer conversion

Sample Input / Output

```
Sample Input:  
aabbbccdddee
```

```
Expected Output:  
Compressed: a2b3c2d4e2  
Decompressed: aabbbccdddee
```

```
For input 'abc':  
Compression not beneficial.  
Original: abc
```

Approach / Design

- **Compression (compress):** traverses the string once, comparing each character to the current run's character and incrementing a count while they match; when the character changes, the finished run is formatted and a new run begins.
- **Run formatting (_format_run):** writes a run of length 1 as just the character (no count), and longer runs as character+count, per the stated constraint.
- **Decompression (decompress):** walks through the encoded string character by character; after each character it accumulates any following digits into a count string, converts it to an integer (defaulting to 1 when no digits follow), and repeats the character that many times.
- **Beneficial-compression check (run_length_encode):** compares the compressed length to the original length and falls back to returning the original string when compression does not actually shorten it (e.g. 'abc').
- **Edge cases:** both compress() and decompress() return an empty string immediately when given empty input, avoiding index errors.

Python Program

```
"""
C02-AT2 - Coding Challenge
Challenge 3: String Compression and Run-Length Encoding (RLE)
[Level: Medium | 15 Marks]
-----
Implements:
(a) compress(s) - replaces consecutive repeated characters with
                  character+count (a run of length 1 is written
                  as just the character, with no count)
(b) decompress(s) - reconstructs the original string from an RLE
                    encoded string
(c) run_length_encode(s) - returns the compressed string, or the
                           original string if compression is not shorter
"""

# -----
# 1. Compression function
# -----
def compress(s):
    """
    Traverses the string once, accumulating a run-length count for
    each group of consecutive identical characters. A run of
    length 1 is written as just the character (no count); longer
    runs are written as character+count.
    """
    if len(s) == 0:
        return ""

    compressed_parts = []
    current_char = s[0]
    count = 1

    for char in s[1:]:
        if char == current_char:
            count += 1
        else:
            compressed_parts.append(_format_run(current_char, count))
            current_char = char
            count = 1
    # append the final run
```

```

        compressed_parts.append(_format_run(current_char, count))

    return "".join(compressed_parts)

def _format_run(char, count):
    """
    Formats a single run: just the character if count is 1,
    otherwise character followed by the count.
    """
    if count == 1:
        return char
    return f"{char}{count}"

# -----
# 2. Decompression function
# -----
def decompress(s):
    """
    Rebuilds the original string from an RLE-encoded string.
    Walks through the string, building up an integer count from
    any consecutive digits following a character; if no digits
    follow, the character's count defaults to 1.
    """
    if len(s) == 0:
        return ""

    result_parts = []
    i = 0

    while i < len(s):
        char = s[i]
        i += 1

        digits = ""
        while i < len(s) and s[i].isdigit():
            digits += s[i]
            i += 1

        count = int(digits) if digits else 1
        result_parts.append(char * count)

    return "".join(result_parts)

# -----
# 3. Run-length encode with "not beneficial" fallback
# -----
def run_length_encode(s):
    """
    Compresses the string using compress(). If the compressed
    string is not shorter than the original, returns the original
    string instead (compression would not be beneficial).
    """
    compressed = compress(s)

    if len(compressed) < len(s):
        return compressed, True
    return s, False

# -----
# 4. Report/display function
# -----
def print_report(s):
    print(f"Original: {s!r}")

    result, was_compressed = run_length_encode(s)

```

```

    if was_compressed:
        print(f"Compressed: {result}")
        rebuilt = decompress(result)
        print(f"Decompressed: {rebuilt}")
        print(f"Round-trip successful: {rebuilt == s}")
    else:
        print("Compression not beneficial.")
        print(f"Original: {result}")

# -----
# 5. Main driver
# -----
def main():
    test_cases = [
        "aabbbccdddee",
        "abc",
        "",
        "aaaa",
        "wwwaaadexxxxxx",
    ]

    for case in test_cases:
        print_report(case)
        print("-" * 50)

if __name__ == "__main__":
    main()

```

Sample Output

```
Original: 'aabbccdddee'  
Compressed: a2b3c2d4e2  
Decompressed: aabbccdddee  
Round-trip successful: True
```

```
-----  
Original: 'abc'  
Compression not beneficial.  
Original: abc
```

```
-----  
Original: ''  
Compression not beneficial.  
Original:
```

```
-----  
Original: 'aaaa'  
Compressed: a4  
Decompressed: aaaa  
Round-trip successful: True
```

```
-----  
Original: 'wwwaaadexxxxx'  
Compressed: w4a3dex6  
Decompressed: wwwaaadexxxxx  
Round-trip successful: True  
-----
```

Conclusion

The program compresses a string in a single pass by accumulating consecutive matching characters into runs, writing single characters without a count as required. Decompression reverses this by reading each character followed by any digit run, defaulting the count to 1 when no digits are present, so it correctly handles RLE strings that mix counted and single-character runs. A length check ensures the original string is returned whenever compressing it would not actually save space. Testing on the sample input, an all single-run string, an empty string, and mixed-length runs all round-trip correctly.